

Universidade Estadual da Paraíba (UEPB)

Disciplina: Programação Web
Professor: Thiago Soares Marques

Laboratório: Construção de API RESTful

Objetivo: Modelar, implementar e testar uma API RESTful completa utilizando o padrão arquitetural MVC, finalizando com o consumo desses dados através de uma interface React.

Exercício 1: Modelagem da API

Toda API precisa de um planejamento ("Contrato"). Vamos criar uma API para gerenciamento de Tarefas (*To-Do List*).

Tarefa 1.1: No seu caderno ou em um arquivo de texto, escreva como será o formato JSON que representa uma única tarefa. Ela deve conter as propriedades: `id`, `titulo` e `concluida`.

Tarefa 1.2: Preencha a tabela abaixo definindo quais verbos HTTP e URLs (*Endpoints*) utilizaremos para cada ação, seguindo as boas práticas do padrão REST:

- Listar todas as tarefas: _____ /api/tarefas
- Criar uma nova tarefa: _____ /api/tarefas
- Buscar uma tarefa específica: _____
- Atualizar uma tarefa: _____
- Deletar uma tarefa: _____

Exercício 2: O Back-End (Model e Controller)

Crie as pastas `models` e `controllers` dentro da pasta `src`.

Tarefa 2.1: No arquivo `src/models/tarefaModel.js`, implemente as funções que manipularão o array de tarefas.

```
1 export let tarefas = []; // Banco de dados em memoria
2
3 export const TarefaModel = {
4   buscarTodas: () => { /* Retorne o array */ },
5   buscarPorId: (id) => { /* Retorne a tarefa correspondente */ },
6   criar: (titulo) => {
7     // 1. Descubra o proximo ID disponivel
8     // 2. Crie a nova tarefa (concluida: false) e adicione ao array
9   },
10  atualizar: (id, concluida) => {
11    // 1. Encontre a tarefa pelo ID
12    // 2. Atualize o status e retorne a tarefa atualizada
13  },
14  deletar: (id) => {
```

```
15     // 1. Encontre o indice da tarefa pelo ID e remova do array
16   }
17 };
```

Tarefa 2.2: No arquivo `src/controllers/tarefaController.js`, crie as funções de validação.

```
1 import { TarefaModel } from '@models/tarefaModel';
2
3 export const adicionarTarefa = (titulo) => {
4   // Verifique se o titulo e valido. Se for, chame TarefaModel.criar
5 };
6
7 export const atualizarStatus = (id, concluida) => {
8   // Verifique se a tarefa existe. Se existir, atualize.
9 };
10 // Implemente listarTarefas, buscarTarefa e removerTarefa
```

Exercício 3: As Rotas e os Testes (Postman)

Tarefa 3.1: Rotas de Coleção

Crie o arquivo `src/app/api/tarefas/route.js`. Implemente as funções GET e POST. Retorne os Status Codes apropriados (200 OK, 201 Created, 400 Bad Request).

```
1 import { NextResponse } from 'next/server';
2 import { adicionarTarefa, listarTarefas } from '@controllers/tarefaController';
3
4 export async function GET() { /* Retorne a lista de tarefas */ }
5 export async function POST(request) { /* Extraia o titulo, valide e
   crie */ }
```

Tarefa 3.2: Rotas Dinâmicas

Crie a pasta e o arquivo `src/app/api/tarefas/[id]/route.js`. O Next.js passará o ID pela variável `params`.

```
1 export async function GET(request, { params }) {
2   // 1. Extraia o id de params e busque a tarefa
3   // 2. Se nao achar, retorne Status 404 (Not Found)
4 }
5 export async function PUT(request, { params }) {
6   // 1. Extraia o id e o novo status (concluida) do request.json()
7   // 2. Atualize via Controller e retorne Status 200
8 }
9 export async function DELETE(request, { params }) {
10  // 1. Delete a tarefa pelo ID e retorne mensagem de sucesso
11 }
```

Tarefa 3.3: O Teste de Fogo

Abra o **Postman** e valide sua API antes de ir para o Front-End:

1. Faça um POST para criar a tarefa "Estudar Next.js".
2. Faça um GET para listar todas e copie o id gerado.
3. Faça um PUT na rota `/api/tarefas/SEU_ID` enviando `{"concluida": true}`.
4. Faça um DELETE na rota `/api/tarefas/SEU_ID`.

Exercício 4: Consumo na Interface (Front-End)

Tarefa 4.1: No arquivo `src/app/page.jsx`, implemente a interface consumindo todos os *endpoints* criados.

```
1 'use client';
2 import { useState, useEffect } from 'react';
3
4 export default function ToDoList() {
5   const [tarefas, setTarefas] = useState([]);
6   const [novoTitulo, setNovoTitulo] = useState('');
7 }
```

```

8 // 1. Implemente o useEffect para fazer o GET inicial e popular o
  estado
9
10 const handleCriar = async (e) => {
11   e.preventDefault();
12   // 2. Faça o fetch (POST), adicione a nova tarefa no estado e limpe
    o input
13 };
14
15 const handleAtualizar = async (id, statusAtual) => {
16   // 3. Faça o fetch (PUT) enviando o status invertido (!statusAtual)
17   // Atualize a tarefa especifica no estado do React
18 };
19
20 const handleDeletar = async (id) => {
21   // 4. Faça o fetch (DELETE) e remova a tarefa do estado visualmente
22 };
23
24 return (
25   <main>
26     <h1>Tarefas</h1>
27     <form onSubmit={handleCriar}>
28       <input value={novoTitulo} onChange={(e) => setNovoTitulo(e.
target.value)} />
29       <button type="submit">Salvar</button>
30     </form>
31     {/* Crie um .map() aqui para exibir as tarefas em formato de
    lista, indicando se ela esta concluida ou nao, e com botoes para "
    Mudar Status" e para "Excluir" a tarefa*/}
32   </main>
33 </main>
34 );
35 }

```